图形合成

> 在双缓冲课程中我们谈到后缓冲区的渲染操作中包含“图形合成”的操作，这篇文章让我们深入了解这个操作底层的机制。

`globalCompositeOperation`

这个字段对于画布的使用者来说并不是一个很常用的属性，一般来说使用默认值 `source-over` 即可。

source-over

这是默认设置，并在现有画布上下文之上绘制新图形。

该字段可以控制画布在绘制新图形时对原有图形采用的遮盖策略。

这其实也就是新旧两个图形如何合成一张图形的问题。

一个半透明的红色像素盖在半透明的绿色像素上是显示什么颜色呢？

我们来想象一个场景，我们拿着一块半透明的红色玻璃和一块半透明的绿色玻璃，它们的透光率都是70%。

> 当光线入射玻璃时，表现有反射、吸收和透射三种性质。光线透过玻璃的性质，称为“透射”，以透光率表示。光线被玻璃阻挡，按一定角度反射出来，称为“反射”，以反射率表示。光线通过玻璃后，一部分光能量被损失，称为“吸收”，以吸收率表示。
>
> 实际到我们眼睛的光其实就是“反射”出来的光。

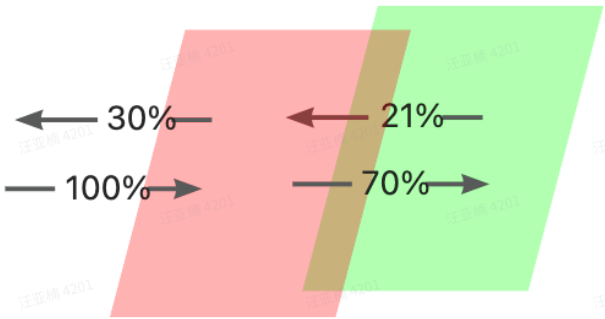
当我们看向一块透光率70%的红色玻璃时，有70%的光透过了玻璃，其他30%的光通过反射进入我们的眼睛里（这里忽略吸收率是因为这是一种模拟的理想情况）。

在计算机中，我们使用 `rgb(255, 0, 0)` 表示这个红色玻璃，而反射到我们眼里的30%的光，也就是它的透明度： `0.3` ， 绿色玻璃同理，所以它们的色值表示如下：

```
red: { r: 255, g: 0, b: 0, a: 0.3 }
green: { r: 0, g: 255, b: 0, a: 0.3 }
```

假如我们看向这块红色玻璃的同时，绿色玻璃紧贴在红色玻璃的正后方，那这时透过红色玻璃的70%的光就会到绿色玻璃上，对于绿色玻璃来说，就有 `70% * 70% = 49%` 的光透过了绿色玻璃，其他 `70% * 30% = 21%` 的光会反射回我们的眼里。

也就是说，处在红色玻璃后的绿色玻璃，能反射到我们眼里的只剩下 `(100% - 30%) * 30%` 的光；



现在我们来计算它们重叠处最终展现的颜色：

① 计算红色玻璃和绿色玻璃反射的色值

红色：

```
const reflectRedColor = {
  r: red.r * red.a,
  g: red.g * red.a,
  b: red.b * red.a
};
// { r: 76.5, g: 0, b: 0 }
console.log(reflectRedColor)
```

绿色：

```
const reflectGreenColor = {
  r: green.r * (1 - red.a) * green.a,
  g: green.g * (1 - red.a) * green.a,
  b: green.b * (1 - red.a) * green.a
```

```
};
// { r: 0, g: 53.55, b: 0 }
console.log(reflectGreenColor)
```

② 红色玻璃和绿色玻璃反射色值的叠加

```
const reflectMixColor = {
  r: red.r * red.a + green.r * (1 - red.a) * green.a,
  g: red.g * red.a + green.g * (1 - red.a) * green.a,
  b: red.b * red.a + green.b * (1 - red.a) * green.a
};
// { r: 76.5, g: 53.55, b: 0 }
console.log(reflectMixColor)
```

③ 计算叠加色的透明度（也就是最后百分之几的光回到眼中）

```
const opacity = red.a + (1 - red.a) * green.a;
// 0.51
console.log(opacity)
```

④ 当前计算出的颜色值是受透明度影响的，需除去透明度的影响

```
// { r: 150, g: 105, b: 0 }
const pureColor = {
  r: reflectMixColor.r / opacity,
  g: reflectMixColor.g / opacity,
  b: reflectMixColor.b / opacity,
}
```

⑤ 最后带上透明度就是叠加色的值（因为舍去了小数，所以是存在误差的）

```
const color = { r: 150, g: 105, b: 0, a: 0.51 }
```

最后效果：

我们上面演示也就是默认图形合成策略：`source-over` 的色值计算流程，这个测试模拟了现实中的颜色光合成。所以实际上当你在画布绘制一个新的图案时，绘制的本质其实就是绘图引擎将像素值进行合成计算，而对于两个图形的合成无非也就是像素一一配对进行合成计算。

合成算法

如果我们将上面的计算过程抽象化可以得到下方的计算公式：

```
const source = { r, g, b, a };
const destination = { r, g, b, a };
const f1 = 1;
const f2 = 1 - a1;
const color = {
  r: source.r * f1 * a1 + destination.r * f2 * a2,
  g: source.g * f1 * a1 + destination.g * f2 * a2,
  b: source.b * f1 * a1 + destination.b * f2 * a2
};
const opacity = a1 * f1 + a2 * f2;
const result = {
  r: color.r / opacity,
  g: color.g / opacity,
  b: color.b / opacity,
  a: opacity
}
```

里面的 `f1`、`f2` 就是控制两个像素的光反射率，如果反过来，让 `f1 = 1 - a2, f2 = 1` 相当于就是将后面的图案放到前面，也就是配置 `destination-over` 的效果。十种合成模式对应的值可看下方配置：

```
/** 十种合成模式 */
const [f1, f2] = {
  'source-over': [1, 1 - a1],
  'source-in': [a2, 0],
  'source-out': [1 - a2, 0],
  'source-atop': [a2, 1 - a1],
  'destination-over': [1 - a2, 1],
  'destination-in': [0, a1],
  'destination-out': [0, 1 - a1],
  'destination-atop': [1 - a2, a1],
  'copy': [1, 0],
  'xor': [1 - a2, 1 - a1]
}[mode];
```

参考

1. Compositing Digital Images
2. Compositing and Blending Level 1